

Chat Socket Contract

This document defines the 1:1 chat socket events, payloads, success responses, and error responses.

Connection and Auth

- Socket auth is handled by `socketAuthMiddleware`.
- Send JWT in `handshake`:
 - Recommended: `Authorization: Bearer <token>`
 - Also supported in query token by `middleware`.

Common Ack Format

All socket events with callback return this format:

- Success:

```
{
  "ok": true,
  "data": {}
}
```

- Error:

```
{
  "ok": false,
  "message": "Error text"
}
```

1) Get or Create Thread

Event

- `chat:thread:getOrCreate`

When to use

- Open chat first time with a user (e.g., from marketplace seller profile).

Emit payload

```
{
  "targetUserId": 12
}
```

Success response (ack)

```
{
  "ok": true,
  "data": {
    "id": 3,
    "userOneId": 2,
    "userTwoId": 12,
    "lastMessageAt": "2026-04-22T10:35:39.000Z",
    "createdAt": "2026-04-22T10:35:39.000Z",
    "updatedAt": "2026-04-22T10:35:39.000Z"
  }
}
```

Error responses (ack)

```
{ "ok": false, "message": "Unauthorized socket user." }
```

```
{ "ok": false, "message": "You cannot create a chat with yourself." }
```

```
{ "ok": false, "message": "Target user not found." }
```

2) Join Thread Room

Event

- `chat:thread:join`

When to use

- Enter chat detail screen before sending/receiving live events.

Emit payload

```
{
  "threadId": 3
}
```

Success response (ack)

```
{
  "ok": true
}
```

Error responses (ack)

```
{ "ok": false, "message": "Unauthorized socket user." }
```

```
{ "ok": false, "message": "Thread not found." }
```

```
{ "ok": false, "message": "You are not a participant of this thread." }
```

3) Send Message

Event

- `chat:message:send`

When to use

- User taps send button.

Emit payload (existing thread)

```
{
  "threadId": 3,
  "message": "Hey! How are you?"
}
```

Emit payload (auto-create thread)

```
{
  "targetUserId": 12,
  "message": "Hey! How are you?"
}
```

Success response (ack)

```
{
  "ok": true,
  "data": {
    "id": 25,
    "threadId": 3,
    "senderId": 2,
    "message": "Hey! How are you?",
    "isRead": false,
    "readAt": null,
    "createdAt": "2026-04-22T10:40:09.000Z",
    "updatedAt": "2026-04-22T10:40:09.000Z",
    "sender": {
      "id": 2,
      "fullName": "Saqib Usman",
      "profileImage": "saqi.png"
    }
  }
}
```

Broadcast event emitted by server

- Event: chat:message:new

```
{
  "threadId": 3,
  "message": {
    "id": 25,
    "threadId": 3,
    "senderId": 2,
    "message": "Hey! How are you?"
  }
}
```

Error responses (ack)

```
{ "ok": false, "message": "Unauthorized socket user." }
```

```
{ "ok": false, "message": "Message is required." }
```

```
{ "ok": false, "message": "threadId or targetUserId is required." }
```

```
{ "ok": false, "message": "Thread not found." }
```

```
{ "ok": false, "message": "You are not a participant of this thread." }
```

4) List Messages

Event

- chat:messages:list

When to use

- Open thread and paginate message history.

Emit payload

```
{
  "threadId": 3,
  "limit": 20,
  "offset": 0
}
```

Success response (ack)

```
{
  "ok": true,
  "data": {
    "count": 42,
    "rows": [
      {
        "id": 25,
        "threadId": 3,
        "senderId": 2,
        "message": "Hey! How are you?",
        "isRead": false,
        "createdAt": "2026-04-22T10:40:09.000Z",
        "sender": {
          "id": 2,
          "fullName": "Saqib Usman",
          "profileImage": "saqi.png"
        }
      }
    ]
  }
}
```

Error responses (ack)

```
{ "ok": false, "message": "Unauthorized socket user." }
```

```
{ "ok": false, "message": "Thread not found." }
```

```
{ "ok": false, "message": "You are not a participant of this thread." }
```

5) List Threads (Chat List Screen)

Event

- chat:threads:list

When to use

- Open chat list tab.

Emit payload

```
{
  "search": "erica",
  "limit": 20,
  "offset": 0
}
```

All payload fields are optional.

Success response (ack)

```
{
  "ok": true,
  "data": {
    "count": 3,
    "rows": [
      {
        "id": 3,
        "userOneId": 2,
        "userTwoId": 12,
        "lastMessageAt": "2026-04-22T10:40:09.000Z",
        "otherUser": {
          "id": 12,
          "fullName": "Erica Alexander",
          "profileImage": "erica.png"
        },
        "lastMessage": {
          "id": 25,
          "threadId": 3,
          "senderId": 2,
          "message": "Hey! How are you?"
        }
      }
    ]
  }
}
```

Notes

- `otherUser` = conversation partner (not current logged-in user).
- `lastMessage` may be null for newly created thread with no messages.

Error responses (ack)

```
{ "ok": false, "message": "Unauthorized socket user." }
```

6) Mark Thread as Read

Event

- `chat:thread:read`

When to use

- User opens thread and reads incoming messages.

Emit payload

```
{
  "threadId": 3
}
```

Success response (ack)

```
{
  "ok": true
}
```

Broadcast event emitted by server

- Event: `chat:thread:read`

```
{
  "threadId": 3,
  "readerId": 2
}
```

Error responses (ack)

```
{ "ok": false, "message": "Unauthorized socket user." }
```

```
{ "ok": false, "message": "Thread not found." }
```

```
{ "ok": false, "message": "You are not a participant of this thread." }
```

Frontend Recommended Sequence

1. Connect socket with JWT auth.

2. Chat list screen:

- `emit chat:threads:list`

3. Open thread:

- `emit chat:thread:join`
- `emit chat:messages:list`
- `emit chat:thread:read`

4. Send message:

- `emit chat:message:send`
- `listen chat:message:new`

5. Start new chat:

- `emit chat:thread:getOrCreate`
- then join and list messages.